MANUAL TECNICO - PROYECTO COMPI FORMS

Fecha: 2026-03-23

Workspace: Proyecto_1_Compi1

Contenido

MANUAL TECNICO - PROYECTO COMPI FORMS 1

1) OBJETIVO DEL PROYECTO 2

2) ESTRUCTURA GENERAL 2

3) TECNOLOGIAS Y VERSIONES CLAVE 2

4) FLUJO TECNICO END-TO-END 3

5) CLASES IMPORTANTES Y COMO FUNCIONAN 4

6) ERRORES POSIBLES Y SOLUCIONES 12

7) COMANDOS RECOMENDADOS DE OPERACION 15

8) CLASES PRINCIPALES (RESUMEN RAPIDO) 16

9) RECOMENDACIONES TECNICAS 17

PARSER 18

PKM_PARSER 44

Lexer 51

PKM_LEXER 58

1) OBJETIVO DEL PROYECTO

Este proyecto permite construir formularios a partir de un lenguaje propio.

El flujo principal es:

1. El usuario escribe codigo .form en la app Android.

2. Se analiza con Lexer + Parser (JFlex + CUP).

3. Se ejecuta validacion semantica.

4. Se construye un FormDocument para renderizar y responder formularios.

5. Opcionalmente se serializa a .pkm y se guarda/carga en un servidor Tomcat.

2) ESTRUCTURA GENERAL

- app/

Cliente Android (UI Compose, parser principal, semantica, render, serializacion PKM y cliente HTTP).

- ApiCompiForms/

Backend Java Jakarta EE (WAR) para Tomcat.

Maneja upload/list/download de archivos .pkm.

- app/src/main/cup y app/src/main/jflex

Definicion de gramatica y lexer.

- app/build/generated/sources/parser

Codigo generado por CUP y JFlex (Parser, Lexer, PKMParser, PKMLexer, etc.).

3) TECNOLOGIAS Y VERSIONES CLAVE

- Android + Kotlin + Jetpack Compose

- Java 21 en modulo app (toolchain)

- JFlex 1.9.1

- Java CUP 11b

- Runtime CUP: com.github.vbmacher:java-cup-runtime

- Backend Jakarta EE 10 en WAR para Tomcat

- ApiCompiForms compilado con Java 17 (pom.xml)

4) FLUJO TECNICO END-TO-END

4.1 Flujo en app (archivo .form)

1. UI captura codigo fuente.

2. analyzeFormCode crea Lexer + Parser.

3. parser.parse() genera arbol/resultado.

4. parser.parseResult regresa ParseResult con:

  - document

  - lexicalErrors

  - syntaxErrors

  - semanticErrors

  - warnings

5. Si isSuccess, se renderiza y se habilitan acciones de guardado/importacion.

4.2 Flujo de guardado PKM

1. Se valida .form actual (analyzeFormCode).

2. Si es valido, PkmSerializer convierte FormDocument a texto .pkm.

3. ApiService.uploadPkmFile envia multipart/form-data a Tomcat.

4. Tomcat guarda el archivo en carpeta de uploads.

4.3 Flujo de importacion PKM

1. ApiService.listPkmFiles consulta archivos.

2. Usuario selecciona uno.

3. ApiService.downloadPkmFile descarga contenido .pkm.

4. PkmDeserializer usa PKMLexer + PKMParser.

5. Si no hay errores de sintaxis PKM, devuelve FormDocument.

6. FormSerializer convierte ese document a codigo legible para editor .form.

5) CLASES IMPORTANTES Y COMO FUNCIONAN

5.1 CAPA UI / ORQUESTACION (APP)

Clase: MainActivity

Responsabilidad:

- Entrada principal Android.

Metodo importante:

+ onCreate(savedInstanceState: Bundle): void

  Configura contenido Compose y monta FormStudioApp.

Clase: FormStudioApp (funcion composable)

Responsabilidad:

- Editor de codigo, analisis, render, import/export PKM, configuracion de servidor.

Metodos/funciones importantes:

+ analyzeFormCode(code: String): ParseResult

  Ejecuta Lexer + Parser del lenguaje .form y retorna resultado completo.

+ generateSectionWithQuestion(type: String): String

  Inserta snippets de ejemplo de preguntas.

Clase: ApiService

Responsabilidad:

- Cliente HTTP para servidor Tomcat y consumo de PokeAPI.

Metodos importantes:

+ getPokemonName(id: Int): String?

  Consulta nombre de Pokemon por id.

+ getPokemonRange(start: Int, end: Int): List<String>

Resuelve rango de nombres con corrutinas.

+ uploadPkmFile(fileName: String, content: String): String

Sube archivo .pkm via multipart.

+ listPkmFiles(): List<String>

Lista archivos disponibles en servidor.

+ downloadPkmFile(fileName: String): String?

Descarga contenido de archivo .pkm.

+ updateServerHost(host: String): void

Cambia host del backend en tiempo de ejecucion.

5.2 ANALISIS LEXICO/SINTACTICO (.form)

Archivo clave: lexer.flex (genera Lexer)

Responsabilidad:

- Tokenizar el lenguaje.
- Registrar tokens para resaltado y errores lexicos.

Metodos clave en clase generada Lexer:

+ getTokens(): List<TokenInfo>

+ getErrors(): List<ErrorInfo>

Relevante:

- Reconoce keywords, operadores, colores, emojis y wildcard.
- Si un simbolo no es reconocido, emite token ERROR y ErrorInfo.

Archivo clave: parser.cup (genera Parser)

Responsabilidad:

- Parsear gramatica del lenguaje.
- Coordinar capa semantica.

Metodos clave en clase generada Parser:

+ getParseResult(): ParseResult

+ getSyntaxErrors(): List<String>

+ getLexer(): Lexer

+ getVariables(): Map<String, Variable<?>>

+ getParsedDocument(): FormDocument

Comportamiento clave:

- Al detectar error de sintaxis, registra expected tokens y posicion.

- En action code inicializa y usa supports semanticos.

Clase: ParseResult

Responsabilidad:

- Resultado unificado del analisis.

Campos clave:

- document

- lexicalErrors

- syntaxErrors

- semanticErrors

- warnings

Propiedad:

+ isSuccess: Boolean

true solo si no hay errores lexicos/sintacticos/semanticos.

5.3 CAPA SEMANTICA

Clase: ParserSemanticSupport

Responsabilidad:

- Coordinador semantico central.

Metodos importantes:

+ addSemanticError(mensaje: String): void

+ addWarning(mensaje: String): void

+ getParseResult(): ParseResult

+ crearResultadoValor(valorReal: Object): Object

+ crearResultadoConComodin(): Object

Clase: SemanticValueSupport

Responsabilidad:

- Conversion de tipos y apoyo para estilos/atributos/listas.

Metodos importantes:

+ convertirADouble(valor: Object): Double

+ convertirATexto(valor: Object): String

+ construirEstilos(valor: Object): StyleSet

+ combinarAtributos(grupo1: Object, grupo2: Object): HashMap<String, Object>

+ validarAtributosObligatorios(...): void

Notas:

- Procesa codigos emoji y secuencias de estrellas.

Clase: VariableSemanticSupport

Responsabilidad:

- Declarar, asignar y resolver variables del lenguaje.

Metodos importantes:

+ declararVariable(nombre, tipo, valorInicial): void

+ asignarVariable(nombre, tipoEsperado, valorNuevo): void

+ asignarVariableSegunTipoDeclarado(nombre, valorNuevo): void

+ resolverValorVariable(nombre): Object

+ prepararVariableFor(nombreVariable): void

Clase: ExpressionSemanticSupport

Responsabilidad:

- Operaciones aritmeticas, comparaciones y logica.

Metodos importantes:

+ evaluarOperacionAritmetica(i, d, operador): Object

+ evaluarComparacion(i, d, operador): Object

+ evaluarOperacionLogica(i, d, operador): Object

+ negarOperacion(valorOriginal): Object

Clase: QuestionSemanticSupport

Responsabilidad:

- Validaciones de preguntas y draw con comodines.

Metodos importantes:

+ crearPreguntaConComodines(pregunta, mapaAtributos): Object

+ validarDraw(nombreVariable, argumentos): void

Clase: LogicalOperatorValidator

Responsabilidad:

- Impedir mezcla inconsistente de operadores logicos.

Metodo importante:

+ validarOperador(operador, support): boolean

5.4 SERIALIZACION / DESERIALIZACION PKM

Clase: PkmSerializer

Responsabilidad:

- Convertir FormDocument en formato .pkm.

Metodo principal:

+ serialize(document, author, description): String

Metodos clave internos:

- generarMetaData(...)

- serializeElement(...)

- serializeStyles(...)

- serializeColor(...)

Punto importante:

- Usa prefijo interno para distinguir TEXT de OPEN durante reconstruccion.

Clase: PkmDeserializer

Responsabilidad:

- Convertir texto .pkm a FormDocument.

Metodos importantes:

+ deserialize(pkmContent): FormDocument?

+ deserializeWithDiagnostics(pkmContent): DeserializeResult

Notas:

- Captura stderr del parser PKM para mensaje de diagnostico.

- Devuelve document null + errorMessage si hay fallo.

Clase: FormSerializer

Responsabilidad:

- Convertir FormDocument a codigo legible para editor .form.

Metodo principal:

+ serialize(document: FormDocument): String

5.5 PARSER PKM

Archivo clave: pkm_lexer.flex (genera PKMLexer)

Responsabilidad:

- Tokenizar sintaxis .pkm.

Archivo clave: pkm_parser.cup (genera PKMParser)

Responsabilidad:

- Construir FormDocument desde .pkm.

Metodos/relevante:

+ hasSyntaxErrors(): boolean

+ syntax_error(Symbol s): void

+ unrecovered_syntax_error(Symbol s): void

5.6 BACKEND TOMCAT (ApiCompiForms)

Clase: BaseApiServlet

Responsabilidad:

- Base de servlets API con utilidades comunes.

Metodos importantes:

setCors(response): void

writeJson(response, status, json): void

jsonEscape(value): String

doOptions(req, resp): void

Clase: HealthServlet

Endpoint:

- GET /api/health

Funcion:

- Health check JSON.

Clase: PkmUploadServlet

Endpoint:

- POST /api/pkm/upload

Funcion:

- Recibe multipart "file" y guarda .pkm.

Clase: PkmFilesServlet

Endpoint:

- GET /api/pkm/files

Funcion:

- Lista archivos .pkm con metadata basica.

Clase: PkmDownloadServlet

Endpoint:

- GET /api/pkm/download/*

Funcion:

- Descarga archivo .pkm por nombre.

Clase: StorageUtil

Responsabilidad:

- Manejo de rutas, sanitizacion y lectura/escritura en disco.

Metodos importantes:

+ getUploadDir(): Path

+ sanitizeFileName(rawName): String

+ resolveTarget(fileName): Path

+ saveUpload(inputStream, fileName): void

+ listPkmFiles(): List<PkmFileInfo>

6) ERRORES POSIBLES Y SOLUCIONES

6.1 Error: Parser/Lexer no existen o estan desactualizados

Sintoma:

- Falla compilacion por clases Parser, Lexer, PKMParser, PKMLexer no encontradas.

Causa:

- No se ejecutaron tareas de generacion o hubo cambio en .cup/.flex.

Solucion:

1. Ejecutar tarea Gradle generateParserAndLexer.
2. Verificar que se genere codigo en build/generated/sources/parser.
3. Recompilar proyecto.

6.2 Error de toolchain Java en app

Sintoma:

- Errores de compilacion por version de Java.

Causa:

- app usa Java 21.

Solucion:

1. Instalar JDK 21.
2. Configurar IDE/Gradle para usar JDK 21.

6.3 Error lexico en .form

Sintoma:

- ParseResult.lexicalErrors con simbolo no reconocido.

Causa:

- Token invalido, typo, simbolo fuera de gramatica.

Solucion:

1. Corregir simbolo reportado y su linea/columna.

2. Verificar operadores validos, comillas y llaves.

6.4 Error sintactico en .form

Sintoma:

- ParseResult.syntaxErrors con token inesperado y expected list.

Causa:

- Estructura incompleta (faltan] , } , , etc.) o orden incorrecto.

Solucion:

1. Revisar token reportado y lo esperado.

2. Corregir delimitadores y orden de atributos.

6.5 Error semantico en variables/operaciones

Sintoma:

- ParseResult.semanticErrors con mensajes de negocio.

Causas comunes:

- Variable no declarada.

- Division por cero.

- Tipo incorrecto.

- Mezcla de operadores logicos incompatible.

Solucion:

1. Declarar variable antes de usarla.

2. Validar operandos numericos.

3. No mezclar OR y AND sin estructura correcta.

4. Revisar draw() y cantidad de comodines.

6.6 Error al serializar a PKM

Sintoma:

- No se puede guardar o el contenido sale incompleto.

Causa:

- FormDocument no valido o datos faltantes.

Solucion:

1. Guardar solo cuando ParseResult.isSuccess sea true.

2. Revisar estilos/atributos opcionales nulos.

6.7 Error al deserializar PKM

Sintoma:

- deserializeWithDiagnostics devuelve document null.

Causa:

- PKM invalido o fuera de la sintaxis esperada por pkm_parser.

Solucion:

1. Mostrar errorMessage al usuario.

2. Validar tags, bloques style/content y metadatos.

3. Regenerar PKM desde serializer oficial si es posible.

6.8 Error de conexion con servidor Tomcat

Sintoma:

- upload/list/download fallan o retornan null/empty.

Causas comunes:

- Host incorrecto.

- Tomcat apagado.

- Context path distinto.

- Firewall/red.

Solucion:

1. Confirmar host configurado en app (ApiService.updateServerHost).

2. Confirmar despliegue en http://HOST:8080/ApiCompiForms.

3. Probar endpoint /api/health.

4. Verificar puertos/firewall.

6.9 Error de subida multipart

Sintoma:

- Respuesta 400 "No se pudo leer multipart" o "Debe enviar un archivo .pkm".

Causa:

- Request mal formado o sin parte file.

Solucion:

1. Verificar nombre de campo multipart: file.
2. Enviar contenido con boundary correcto.

6.10 Error de seguridad/path traversal en nombre de archivo

Sintoma:

- Falla al resolver destino o nombre cambiado.

Causa:

- Nombre con caracteres invalidos o rutas embebidas.

Solucion:

1. Usar sanitizeFileName.
2. No permitir rutas relativas/absolutas en nombre.

7) COMANDOS RECOMENDADOS DE OPERACION

7.1 App Android

- Construir: ./gradlew :app:assembleDebug
- Tests unitarios: ./gradlew :app:testDebugUnitTest

7.2 Generar parser/lexer

- ./gradlew :app:generateParserAndLexer

7.3 Backend Tomcat (ApiCompiForms)

- Empaquetar WAR: mvn clean package

- Desplegar WAR en Tomcat (webapps)

8) CLASES PRINCIPALES (RESUMEN RAPIDO)

- MainActivity / FormStudioApp

  Interfaz principal y orquestacion de analisis y guardado.

- Lexer / Parser

  Analisis lexico y sintactico del lenguaje .form.

- ParserSemanticSupport + supports especializados

  Reglas semanticas: tipos, variables, expresiones, validaciones de preguntas.

- ParseResult

  Contrato de salida del analizador.

- PkmSerializer / PkmDeserializer / FormSerializer

  Conversion bidireccional entre FormDocument, .pkm y texto de editor.

- ApiService

  Cliente HTTP para servidor Tomcat y PokeAPI.

- BaseApiServlet + PkmUploadServlet + PkmFilesServlet + PkmDownloadServlet

  Backend de archivos .pkm.

- StorageUtil

  Utilidades de almacenamiento y sanitizacion de archivos.

9) RECOMENDACIONES TECNICAS

1. Mantener sincronizados .cup/.flex y fuentes generadas.
2. No editar manualmente clases generadas en build/generated.
3. Validar siempre ParseResult.isSuccess antes de exportar/subir.
4. Centralizar mensajes de error para UX consistente.
5. Mantener pruebas de round-trip:
   - form -> FormDocument -> pkm -> FormDocument -> form.
6. Versionar cambios de gramatica junto con ejemplos de entrada/salida.

PARSER

```
package com.cunoc.compiforms;

import com.cunoc.compiforms.Lexer;
import com.cunoc.compiforms.sym;
import com.cunoc.compiforms.data.*;
import com.cunoc.compiforms.form.model.*;
import com.cunoc.compiforms.form.model.elements.*;
import com.cunoc.compiforms.form.model.questions.*;
import com.cunoc.compiforms.form.model.styles.*;
import com.cunoc.compiforms.tokens.*;
import java_cup.runtime.Symbol;
import java.util.*;
import com.cunoc.compiforms.semantic.ParserSemanticSupport;
import com.cunoc.compiforms.semantic.SemanticValueSupport;
import com.cunoc.compiforms.semantic.VariableSemanticSupport;
import com.cunoc.compiforms.semantic.ExpressionSemanticSupport;
import com.cunoc.compiforms.semantic.QuestionSemanticSupport;

parser code {:

Lexer lexer;
private List<String> syntaxErrors;

// conexion de parser y lexer
public Parser(Lexer lex) {
   super(lex);
   this.lexer = lex;
   syntaxErrors = new ArrayList<>();
}

public Lexer getLexer() {
   return this.lexer;
}

public List<String> getSyntaxErrors() {
   return this.syntaxErrors;
}

public ParseResult getParseResult() {
  return ((CUP$Parser$actions) action_obj).getParseResult();
}

public Map<String, Variable<?>> getVariables() {
```

```
  return ((CUP$Parser$actions) action_obj).getVariables();
}

public FormDocument getParsedDocument() {
  return ((CUP$Parser$actions) action_obj).getParsedDocument();
}

public List<String> getSemanticErrors() {
  return ((CUP$Parser$actions) action_obj).getSemanticErrors();
}

public List<String> getWarnings() {
  return ((CUP$Parser$actions) action_obj).getWarnings();
}

private String nombreSimboloDesdeId(int tokenId) {
  if (tokenId >= 0 && tokenId < sym.terminalNames.length) {
    return sym.terminalNames[tokenId];
  }
  return "TOKEN_DESCONOCIDO(" + tokenId + ")";
}

private void customHandleError(Symbol cur_token) {
    StringBuilder mssBuilder = new StringBuilder();
    mssBuilder.append(nombreSimboloDesdeId(cur_token.sym));
    mssBuilder.append(", line: ").append(cur_token.left);
    mssBuilder.append(", col: ").append(cur_token.right);

    if(expected_token_ids().isEmpty() ) {
        mssBuilder.append(" -- ya no se esperaba ningun simbolo");
    } else {
        mssBuilder.append(" se esperaba: ");
        for(Integer expected_token_id : expected_token_ids() ) {
            mssBuilder.append(nombreSimboloDesdeId(expected_token_id)).append(" ");
        }
    }
    syntaxErrors.add(mssBuilder.toString());
}

public void syntax_error(Symbol cur_token) {
    customHandleError(cur_token);
}

public void report_error(String message, Object info) {
```

```
    try {
        Symbol cur_token = (Symbol) info;
        customHandleError(cur_token);
    } catch (Exception e) {
        syntaxErrors.add(message);
    }
}

public void unrecovered_syntax_error(Symbol cur_token) {
    syntaxErrors.add("Error fatal: no se puede recuperar del error de sintaxis");
}

:};

action code {:
  private ParserSemanticSupport semanticSupport;
  private SemanticValueSupport valueSupport;
  private VariableSemanticSupport variableSupport;
  private ExpressionSemanticSupport expressionSupport;
  private QuestionSemanticSupport questionSupport;

  private void initSemanticSupports() {
    if (semanticSupport != null) return;

    semanticSupport = new ParserSemanticSupport(parser);
    valueSupport = new SemanticValueSupport(semanticSupport);
    variableSupport = new VariableSemanticSupport(semanticSupport.getVariables(),
valueSupport, semanticSupport);
    expressionSupport = new ExpressionSemanticSupport(valueSupport, semanticSupport);
    questionSupport = new QuestionSemanticSupport(semanticSupport.getVariables(),
valueSupport, semanticSupport);
  }

  private ParserSemanticSupport semanticSupport() {
    initSemanticSupports();
    return semanticSupport;
  }

  private SemanticValueSupport valueSupport() {
    initSemanticSupports();
    return valueSupport;
  }

  private VariableSemanticSupport variableSupport() {
```

```
    initSemanticSupports();
    return variableSupport;
  }

  private ExpressionSemanticSupport expressionSupport() {
    initSemanticSupports();
    return expressionSupport;
  }

  private QuestionSemanticSupport questionSupport() {
    initSemanticSupports();
    return questionSupport;
  }

  public Map<String, Variable<?>> getVariables() { return semanticSupport().getVariables(); }
  public FormDocument getParsedDocument() { return
semanticSupport().getParsedDocument(); }
  public List<String> getSemanticErrors() { return semanticSupport().getSemanticErrors(); }
  public List<String> getWarnings() { return semanticSupport().getWarnings(); }
  public ParseResult getParseResult() { return semanticSupport().getParseResult(); }

  private FormDocument registrarDocumento(FormDocument documento) {
semanticSupport().setParsedDocument(documento); return documento; }

  // Delegación a través del "experto en Valores"
  private HashMap<String, Object> crearAtributo(String n, Object v) { return
valueSupport().crearAtributo(n, v); }
  private HashMap<String, Object> combinarAtributos(Object i, Object d) { return
valueSupport().combinarAtributos(i, d); }
  private Double convertirADouble(Object v) { return valueSupport().convertirADouble(v); }
  private Integer convertirAEntero(Object v) { return valueSupport().convertirAEntero(v); }
  private String convertirATexto(Object v) { return valueSupport().convertirATexto(v); }
  private String normalizarLiteralCadena(Object v) { return
valueSupport().normalizarLiteralCadena(v); }
  private Orientacion convertirAOrientacion(Object v) { return
valueSupport().convertirAOrientacion(v); }
  private FontFamily convertirAFuente(Object v) { return valueSupport().convertirAFuente(v); }
  private BorderType convertirATipoBorde(Object v) { return
valueSupport().convertirATipoBorde(v); }
  private ColorValue crearColorNombrado(Object v) { return
valueSupport().crearColorNombrado(v); }
  private ColorValue crearColorDesdeTexto(Object v) { return
valueSupport().crearColorDesdeTexto(v); }
  private BorderStyle crearEstiloBorde(Object s, Object t, Object c) { return
```

```
valueSupport().crearEstiloBorde(s, t, c); }
  private StyleSet construirEstilos(Object v) { return valueSupport().construirEstilos(v); }
  private ArrayList<FormElement> extraerElementosFormulario(Object v) { return
valueSupport().extraerElementosFormulario(v); }
  private ArrayList<TableRow> extraerFilasTabla(Object v) { return
valueSupport().extraerFilasTabla(v); }
  private ArrayList<String> extraerListaTextos(Object v) { return
valueSupport().extraerListaTextos(v); }
  private ArrayList<Integer> extraerListaEnteros(Object v) { return
valueSupport().extraerListaEnteros(v); }
  private ArrayList<Object> extraerListaObjetos(Object v) { return
valueSupport().extraerListaObjetos(v); }
  private void validarAtributosObligatorios(Map<String, Object> m, String n, String[] r) {
valueSupport().validarAtributosObligatorios(m, n, r); }

  // Delegación a través del "experto en Variables"
  private void declararVariable(String n, VariableType t, Object v) {
variableSupport().declararVariable(n, t, v); }
  private void asignarVariable(String n, VariableType t, Object v) {
variableSupport().asignarVariable(n, t, v); }
  private void asignarVariableSegunTipoDeclarado(String n, Object v) {
variableSupport().asignarVariableSegunTipoDeclarado(n, v); }
  private void prepararVariableFor(String n) { variableSupport().prepararVariableFor(n); }
  private Object resolverValorVariable(String n) { return
variableSupport().resolverValorVariable(n); }

  // Delegación a través del "experto en Expresiones"
  private Object evaluarOperacionAritmetica(Object i, Object d, String o) { return
expressionSupport().evaluarOperacionAritmetica(i, d, o); }
  private Object evaluarComparacion(Object i, Object d, String o) { return
expressionSupport().evaluarComparacion(i, d, o); }
  private Object evaluarOperacionLogica(Object i, Object d, String o) { return
expressionSupport().evaluarOperacionLogica(i, d, o); }
  private Object negarOperacionLogica(Object v) { return
expressionSupport().negarOperacionLogica(v); }

  // Delegación a través del "experto en Preguntas"
  private Object crearPreguntaConComodines(QuestionElement q, Object m) { return
questionSupport().crearPreguntaConComodines(q, m); }
  private void validarDraw(String n, Object a) { questionSupport().validarDraw(n, a); }
  private void validarIndiceCorrecto(String n, Integer i, int t) {
questionSupport().validarIndiceCorrecto(n, i, t); }
  private void validarIndicesCorrectos(String n, List<Integer> i, int t) {
questionSupport().validarIndicesCorrectos(n, i, t); }
```

```
  private void validarCantidadOpcionesSelect(List<String> o) {
questionSupport().validarCantidadOpcionesSelect(o); }

  // Utilidades generales del Soporte
  private Object crearResultadoValor(Object v) { return
semanticSupport().crearResultadoValor(v); }
  private Object crearResultadoConComodin() { return
semanticSupport().crearResultadoConComodin(); }
  private QuestionElement extraerPreguntaElemento(Object v) { return
semanticSupport().extraerPreguntaElemento(v); }
:};

/* ==================================================
TERMINALES
================================================== */

terminal SECTION, TABLE, TEXT;
terminal OPEN_QUESTION, DROP_QUESTION, SELECT_QUESTION, MULTIPLE_QUESTION;
terminal IF, ELSE, WHILE, DO, FOR, IN;
terminal WIDTH, HEIGHT, LABEL, ORIENTATION, ELEMENTS, POINTX, POINTY, CONTENT;
terminal COLOR, BACKGROUND_COLOR, FONT_FAMILY, TEXT_SIZE, BORDER;
terminal OPTIONS, CORRECT, STYLES;
terminal NUMBER, STRING, SPECIAL;
terminal VERTICAL, HORIZONTAL;
terminal MONO, SANS_SERIF, CURSIVE;
terminal LINE, DOTTED, DOUBLE;
terminal RED, BLUE, GREEN, PURPLE, SKY, YELLOW, BLACK, WHITE;
terminal WHO_IS_THAT_POKEMON, DRAW;
terminal MAS, MENOS, POR, DIV, POTENCIA, MODULO;
terminal UMINUS;
terminal PAREN_IZQ, PAREN_DER;
terminal IGUAL, DIFERENTE, MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL;
terminal AND, OR, NOT;
terminal ASIGNACION;
terminal LLAVE_IZQ, LLAVE_DER, COR_IZQ, COR_DER;
terminal DOS_PUNTOS, COMA, PUNTO, RANGO, PUNTOYCOMA;
terminal WILDCARD;
terminal ENTERO, DECIMAL, CADENA, ID, EMOJI;
terminal HEX_COLOR, RGB_COLOR, HSL_COLOR;
terminal ERROR;


/* ==================================================
NO TERMINALES
```

```
================================================== */

non terminal programa;
non terminal lista_instrucciones;
non terminal instruccion;

non terminal declarar_variable;
non terminal asignacion;
non terminal asignacion_numerica;
non terminal llamada_draw;

non terminal seccion;
non terminal tabla;
non terminal texto;
non terminal pregunta;

non terminal lista_argumentos;

non terminal lista_elementos;
non terminal elemento;

non terminal filas;
non terminal fila;
non terminal lista_celdas;
non terminal celda;

non terminal lista_opciones_min2;
non terminal lista_numeros;
non terminal argumento_pokemon;

non terminal atributos_section;
non terminal atributos_tabla;
non terminal atributos_texto;
non terminal atributos_open_pregunta;
non terminal atributos_drop_pregunta;
non terminal atributos_select_pregunta;
non terminal atributos_multiple_pregunta;

non terminal atributo_section;
non terminal atributo_tabla;
non terminal atributo_texto;
non terminal atributo_open_pregunta;
non terminal atributo_drop_pregunta;
non terminal atributo_select_pregunta;
```

```
non terminal atributo_multiple_pregunta;

non terminal no_terminal_orientacion;

non terminal lista_estilos;
non terminal estilo;
non terminal bordes;
non terminal fuentes;
non terminal colores;

non terminal bloque_if;
non terminal bloque_else;
non terminal bloque_while;
non terminal bloque_do;
non terminal bloque_for;

non terminal expresion;
non terminal expresion_logica;
non terminal expresion_logica_or;
non terminal expresion_logica_and;
non terminal expresion_comparacion;
non terminal expresion_aritmetica;
non terminal valor;

non terminal cadena_con_emojis;


/* ================================================
PRECEDENCIA
================================================ */

precedence left OR;
precedence left AND;
precedence right NOT;

precedence left IGUAL, DIFERENTE, MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL;

precedence left MAS, MENOS;
precedence left POR, DIV;
precedence left POTENCIA, MODULO;

precedence right UMINUS;
```

```
/* ================================================
INICIO
================================================ */

start with programa;

programa ::= lista_instrucciones:listaInstruccionesReducida
{:
  ArrayList<FormElement> elementosRaiz =
extraerElementosFormulario(listaInstruccionesReducida);
  RESULT = registrarDocumento(new FormDocument(elementosRaiz));
:}
;


/* ================================================
INSTRUCCIONES
================================================ */

lista_instrucciones ::= lista_instrucciones:listaAcumulada instruccion:instruccionReducida
                {:
                  ArrayList<FormElement> elementosAcumulados =
extraerElementosFormulario(listaAcumulada);
                  if (instruccionReducida instanceof FormElement)
elementosAcumulados.add((FormElement) instruccionReducida);
                  RESULT = elementosAcumulados;
                :}
                | instruccion:instruccionReducida
                {:
                  ArrayList<FormElement> elementosIniciales = new ArrayList<>();
                  if (instruccionReducida instanceof FormElement)
elementosIniciales.add((FormElement) instruccionReducida);
                  RESULT = elementosIniciales;
                :}
;

instruccion ::=
    declarar_variable
  {: RESULT = null; :}
   | asignacion
  {: RESULT = null; :}
   | llamada_draw
  {: RESULT = null; :}
   | seccion:s
```

```
    {: RESULT = s; :}
   | tabla:t
    {: RESULT = t; :}
   | bloque_if
  {: RESULT = null; :}
   | bloque_while
  {: RESULT = null; :}
   | bloque_do
  {: RESULT = null; :}
   | bloque_for
  {: RESULT = null; :}
   | ERROR
  {: RESULT = null; :}
;


/* ================================================
VARIABLES
================================================ */

declarar_variable ::=
    NUMBER ID:id
    {:
      declararVariable(id.toString(), VariableType.NUMBER, 0.0);
    :}
   | STRING ID:id
   {:
    declararVariable(id.toString(), VariableType.STRING, "");
   :}
   | NUMBER ID:id ASIGNACION expresion_aritmetica:valorInicial
    {:
      declararVariable(id.toString(), VariableType.NUMBER, valorInicial);
    :}
   | STRING ID:id ASIGNACION cadena_con_emojis:valorInicial
    {:
      declararVariable(id.toString(), VariableType.STRING, valorInicial);
    :}
   | SPECIAL ID:id ASIGNACION pregunta:preguntaEspecial
    {:
      declararVariable(id.toString(), VariableType.SPECIAL, preguntaEspecial);
    :}
;

asignacion ::= ID:id ASIGNACION expresion_aritmetica:valorAsignado
```

```
{:
  asignarVariableSegunTipoDeclarado(id.toString(), valorAsignado);
:}
;

asignacion ::= ID:id ASIGNACION pregunta:preguntaEspecial
{:
  asignarVariable(id.toString(), VariableType.SPECIAL, preguntaEspecial);
:}
;

asignacion_numerica ::= ID:id ASIGNACION expresion_aritmetica:valorAsignado
{:
  prepararVariableFor(id.toString());
  asignarVariable(id.toString(), VariableType.NUMBER, valorAsignado);
:}
;


/* ==================================================
DRAW
================================================== */

llamada_draw ::= ID:id PUNTO DRAW PAREN_IZQ lista_argumentos:argumentosDraw
PAREN_DER
{:
  validarDraw(id.toString(), argumentosDraw);
:}
;

lista_argumentos ::= lista_argumentos:argumentosAcumulados COMA valor:valorReducido
            {:
              ArrayList<Object> listaArgumentos = extraerListaObjetos(argumentosAcumulados);
              listaArgumentos.add(valorReducido);
              RESULT = listaArgumentos;
            :}
            | valor:valorReducido
            {:
              ArrayList<Object> listaArgumentos = new ArrayList<>();
              listaArgumentos.add(valorReducido);
              RESULT = listaArgumentos;
            :}
            |
            {:
```

```
                RESULT = new ArrayList<Object>();
               :}
;


/* ================================================
SECTION
================================================ */

seccion ::= SECTION COR_IZQ atributos_section:atributosSeccion COR_DER
{:
  HashMap<String, Object> mapaAtributosSeccion = combinarAtributos(atributosSeccion,
null);
  validarAtributosObligatorios(mapaAtributosSeccion, "SECTION", new String[]{"width",
"height", "pointX", "pointY"});
  LayoutProps propiedadesLayout = new LayoutProps(
    convertirADouble(mapaAtributosSeccion.get("width")),
    convertirADouble(mapaAtributosSeccion.get("height")),
    convertirADouble(mapaAtributosSeccion.get("pointX")),
    convertirADouble(mapaAtributosSeccion.get("pointY"))
  );
  Orientacion orientacionSeccion =
convertirAOrientacion(mapaAtributosSeccion.get("orientation"));
  ArrayList<FormElement> elementosInternos =
extraerElementosFormulario(mapaAtributosSeccion.get("elements"));
  StyleSet estilosSeccion = construirEstilos(mapaAtributosSeccion.get("styles"));
  RESULT = new SectionElement(propiedadesLayout, orientacionSeccion, elementosInternos,
estilosSeccion);
:}
;

atributos_section ::= atributos_section:atributosAcumulados COMA
atributo_section:atributoActual
                {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
                | atributo_section:atributoActual
                {: RESULT = atributoActual; :}
;

atributo_section ::=
     WIDTH DOS_PUNTOS expresion:v
     {: RESULT = crearAtributo("width", v); :}
   | HEIGHT DOS_PUNTOS expresion:v
     {: RESULT = crearAtributo("height", v); :}
   | POINTX DOS_PUNTOS expresion:v
```

```
    {: RESULT = crearAtributo("pointX", v); :}
   | POINTY DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("pointY", v); :}
   | ORIENTATION DOS_PUNTOS no_terminal_orientacion:v
    {: RESULT = crearAtributo("orientation", v); :}
   | ELEMENTS DOS_PUNTOS LLAVE_IZQ lista_elementos:v LLAVE_DER
    {: RESULT = crearAtributo("elements", v); :}
   | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;


/* ==================================================
TABLE
================================================== */

tabla ::= TABLE COR_IZQ atributos_tabla:atributosTabla COR_DER
{:
  HashMap<String, Object> mapaAtributosTabla = combinarAtributos(atributosTabla, null);
  validarAtributosObligatorios(mapaAtributosTabla, "TABLE", new String[]{"width", "height",
"pointX", "pointY"});
  LayoutProps propiedadesLayout = new LayoutProps(
    convertirADouble(mapaAtributosTabla.get("width")),
    convertirADouble(mapaAtributosTabla.get("height")),
    convertirADouble(mapaAtributosTabla.get("pointX")),
    convertirADouble(mapaAtributosTabla.get("pointY"))
  );
  ArrayList<TableRow> filasTabla = extraerFilasTabla(mapaAtributosTabla.get("rows"));
  StyleSet estilosTabla = construirEstilos(mapaAtributosTabla.get("styles"));
  RESULT = new TableElement(propiedadesLayout, filasTabla, estilosTabla);
:}
;

atributos_tabla ::= atributos_tabla:atributosAcumulados COMA atributo_tabla:atributoActual
            {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
            | atributo_tabla:atributoActual
            {: RESULT = atributoActual; :}
;

atributo_tabla ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
   | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
```

```
   | POINTX DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("pointX", v); :}
   | POINTY DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("pointY", v); :}
   | ELEMENTS DOS_PUNTOS LLAVE_IZQ filas:v LLAVE_DER
    {: RESULT = crearAtributo("rows", v); :}
   | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;


/* ==================================================
TEXT
================================================== */

texto ::= TEXT COR_IZQ atributos_texto:atributosTexto COR_DER
{:
  HashMap<String, Object> mapaAtributosTexto = combinarAtributos(atributosTexto, null);
  LayoutProps propiedadesLayout = new LayoutProps(
    convertirADouble(mapaAtributosTexto.get("width")),
    convertirADouble(mapaAtributosTexto.get("height")),
    convertirADouble(mapaAtributosTexto.get("pointX")),
    convertirADouble(mapaAtributosTexto.get("pointY"))
  );
  StyleSet estilosTexto = construirEstilos(mapaAtributosTexto.get("styles"));
  RESULT = new TextElement(propiedadesLayout,
convertirATexto(mapaAtributosTexto.get("content")), estilosTexto);
:}
;

atributos_texto ::= atributos_texto:atributosAcumulados COMA atributo_texto:atributoActual
            {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
            | atributo_texto:atributoActual
            {: RESULT = atributoActual; :}
;

atributo_texto ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
   | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
   | POINTX DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("pointX", v); :}
   | POINTY DOS_PUNTOS expresion:v
```

```
    {: RESULT = crearAtributo("pointY", v); :}
  | CONTENT DOS_PUNTOS cadena_con_emojis:v
    {: RESULT = crearAtributo("content", v); :}
  | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;


/* ==================================================
PREGUNTAS
================================================== */

pregunta ::=
    OPEN_QUESTION COR_IZQ atributos_open_pregunta:atributosOpen COR_DER
    {:
      HashMap<String, Object> mapaAtributosOpen = combinarAtributos(atributosOpen, null);
      validarAtributosObligatorios(mapaAtributosOpen, "OPEN_QUESTION", new
String[]{"label"});
      LayoutProps propiedadesLayout = new
LayoutProps(convertirADouble(mapaAtributosOpen.get("width")),
convertirADouble(mapaAtributosOpen.get("height")), null, null);
      StyleSet estilosPregunta = construirEstilos(mapaAtributosOpen.get("styles"));
      OpenQuestionElement preguntaAbierta = new
OpenQuestionElement(propiedadesLayout,
convertirATexto(mapaAtributosOpen.get("label")), estilosPregunta);
      RESULT = crearPreguntaConComodines(preguntaAbierta, mapaAtributosOpen);
    :}
  | DROP_QUESTION COR_IZQ atributos_drop_pregunta:atributosDrop COR_DER
    {:
      HashMap<String, Object> mapaAtributosDrop = combinarAtributos(atributosDrop, null);
      validarAtributosObligatorios(mapaAtributosDrop, "DROP_QUESTION", new
String[]{"label", "optionsSource"});
      LayoutProps propiedadesLayout = new
LayoutProps(convertirADouble(mapaAtributosDrop.get("width")),
convertirADouble(mapaAtributosDrop.get("height")), null, null);
      Object origenOpcionesCrudo = mapaAtributosDrop.get("optionsSource");
      OptionsSource fuenteOpciones = origenOpcionesCrudo instanceof OptionsSource ?
(OptionsSource) origenOpcionesCrudo : new OptionsSource.Static(new ArrayList<>());
      Integer indiceCorrecto = convertirAEntero(mapaAtributosDrop.get("correct"));
      int totalOpciones = fuenteOpciones instanceof OptionsSource.Static ?
((OptionsSource.Static) fuenteOpciones).getOptions().size() :
(((OptionsSource.PokemonRange) fuenteOpciones).getEnd() -
((OptionsSource.PokemonRange) fuenteOpciones).getStart()) + 1;
      validarIndiceCorrecto("DROP_QUESTION", indiceCorrecto, totalOpciones);
```

```
    StyleSet estilosPregunta = construirEstilos(mapaAtributosDrop.get("styles"));
    DropQuestionElement preguntaDesplegable = new
DropQuestionElement(propiedadesLayout, convertirATexto(mapaAtributosDrop.get("label")),
fuenteOpciones, indiceCorrecto == null ? -1 : indiceCorrecto, estilosPregunta);
    RESULT = crearPreguntaConComodines(preguntaDesplegable, mapaAtributosDrop);
   :}
  | SELECT_QUESTION COR_IZQ atributos_select_pregunta:atributosSelect COR_DER
   {:
    HashMap<String, Object> mapaAtributosSelect = combinarAtributos(atributosSelect,
null);
    validarAtributosObligatorios(mapaAtributosSelect, "SELECT_QUESTION", new
String[]{"options"});
    LayoutProps propiedadesLayout = new
LayoutProps(convertirADouble(mapaAtributosSelect.get("width")),
convertirADouble(mapaAtributosSelect.get("height")), null, null);
    ArrayList<String> opcionesPregunta =
extraerListaTextos(mapaAtributosSelect.get("options"));
    Integer indiceCorrecto = convertirAEntero(mapaAtributosSelect.get("correct"));
    validarIndiceCorrecto("SELECT_QUESTION", indiceCorrecto, opcionesPregunta.size());
    validarCantidadOpcionesSelect(opcionesPregunta);
    StyleSet estilosPregunta = construirEstilos(mapaAtributosSelect.get("styles"));
    SelectQuestionElement preguntaSeleccion = new
SelectQuestionElement(propiedadesLayout, opcionesPregunta, indiceCorrecto,
estilosPregunta);
    RESULT = crearPreguntaConComodines(preguntaSeleccion, mapaAtributosSelect);
   :}
  | MULTIPLE_QUESTION COR_IZQ atributos_multiple_pregunta:atributosMultiple COR_DER
   {:
    HashMap<String, Object> mapaAtributosMultiple = combinarAtributos(atributosMultiple,
null);
    validarAtributosObligatorios(mapaAtributosMultiple, "MULTIPLE_QUESTION", new
String[]{"options"});
    LayoutProps propiedadesLayout = new
LayoutProps(convertirADouble(mapaAtributosMultiple.get("width")),
convertirADouble(mapaAtributosMultiple.get("height")), null, null);
    ArrayList<String> opcionesPregunta =
extraerListaTextos(mapaAtributosMultiple.get("options"));
    ArrayList<Integer> indicesCorrectos =
extraerListaEnteros(mapaAtributosMultiple.get("correctList"));
    validarIndicesCorrectos("MULTIPLE_QUESTION", indicesCorrectos,
opcionesPregunta.size());
    StyleSet estilosPregunta = construirEstilos(mapaAtributosMultiple.get("styles"));
    MultipleQuestionElement preguntaMultiple = new
MultipleQuestionElement(propiedadesLayout, opcionesPregunta, indicesCorrectos,
```

```
estilosPregunta);
      RESULT = crearPreguntaConComodines(preguntaMultiple, mapaAtributosMultiple);
    :}
;

atributos_open_pregunta ::= atributos_open_pregunta:atributosAcumulados COMA
atributo_open_pregunta:atributoActual
                {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
                | atributo_open_pregunta:atributoActual
                {: RESULT = atributoActual; :}
;

atributo_open_pregunta ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
   | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
   | LABEL DOS_PUNTOS cadena_con_emojis:v
    {: RESULT = crearAtributo("label", v); :}
   | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;

atributos_drop_pregunta ::= atributos_drop_pregunta:atributosAcumulados COMA
atributo_drop_pregunta:atributoActual
                {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
                | atributo_drop_pregunta:atributoActual
                {: RESULT = atributoActual; :}
;

atributo_drop_pregunta ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
   | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
   | LABEL DOS_PUNTOS cadena_con_emojis:v
    {: RESULT = crearAtributo("label", v); :}
   | OPTIONS DOS_PUNTOS LLAVE_IZQ lista_opciones_min2:v LLAVE_DER
    {: RESULT = crearAtributo("optionsSource", new
OptionsSource.Static(extraerListaTextos(v))); :}
   | OPTIONS DOS_PUNTOS WHO_IS_THAT_POKEMON PAREN_IZQ
argumento_pokemon:selector COMA ENTERO:inicio COMA ENTERO:end PAREN_DER
    {:
      String selectorRango = convertirATexto(selector);
```

```
      int inicioRango = convertirAEntero(inicio) == null ? 0 : convertirAEntero(inicio);
      int finRango = convertirAEntero(end) == null ? 0 : convertirAEntero(end);
      RESULT = crearAtributo("optionsSource", new
OptionsSource.PokemonRange(selectorRango, inicioRango, finRango));
    :}
  | CORRECT DOS_PUNTOS expresion_aritmetica:v
    {: RESULT = crearAtributo("correct", v); :}
  | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;

atributos_select_pregunta ::= atributos_select_pregunta:atributosAcumulados COMA
atributo_select_pregunta:atributoActual
                    {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
                    | atributo_select_pregunta:atributoActual
                    {: RESULT = atributoActual; :}
;

atributo_select_pregunta ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
  | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
  | OPTIONS DOS_PUNTOS LLAVE_IZQ lista_opciones_min2:v LLAVE_DER
    {: RESULT = crearAtributo("options", v); :}
  | CORRECT DOS_PUNTOS expresion_aritmetica:v
    {: RESULT = crearAtributo("correct", v); :}
  | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;

atributos_multiple_pregunta ::= atributos_multiple_pregunta:atributosAcumulados COMA
atributo_multiple_pregunta:atributoActual
                     {: RESULT = combinarAtributos(atributosAcumulados, atributoActual); :}
                     | atributo_multiple_pregunta:atributoActual
                     {: RESULT = atributoActual; :}
;

atributo_multiple_pregunta ::=
    WIDTH DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("width", v); :}
  | HEIGHT DOS_PUNTOS expresion:v
    {: RESULT = crearAtributo("height", v); :}
  | OPTIONS DOS_PUNTOS LLAVE_IZQ lista_opciones_min2:v LLAVE_DER
```

```
    {: RESULT = crearAtributo("options", v); :}
  | CORRECT DOS_PUNTOS LLAVE_IZQ lista_numeros:v LLAVE_DER
    {: RESULT = crearAtributo("correctList", v); :}
  | STYLES COR_IZQ lista_estilos:v COR_DER
    {: RESULT = crearAtributo("styles", v); :}
;

no_terminal_orientacion ::= VERTICAL:v {: RESULT = v; :}
          | HORIZONTAL:v {: RESULT = v; :}
;


/* ==================================================
ESTILOS
================================================== */

lista_estilos ::= lista_estilos:estilosAcumulados COMA estilo:estiloActual
            {: RESULT = combinarAtributos(estilosAcumulados, estiloActual); :}
            | estilo:estiloActual
            {: RESULT = estiloActual; :}
;

estilo ::=
    COLOR DOS_PUNTOS colores:colorTexto
    {: RESULT = crearAtributo("textColor", colorTexto); :}
  | BACKGROUND_COLOR DOS_PUNTOS colores:colorFondo
    {: RESULT = crearAtributo("backgroundColor", colorFondo); :}
  | FONT_FAMILY DOS_PUNTOS fuentes:fuenteTexto
    {: RESULT = crearAtributo("fontFamily", fuenteTexto); :}
  | TEXT_SIZE DOS_PUNTOS expresion:tamanioTexto
    {: RESULT = crearAtributo("textSize", tamanioTexto); :}
  | BORDER DOS_PUNTOS PAREN_IZQ expresion:tamanioBorde COMA bordes:tipoBorde
COMA colores:colorBorde PAREN_DER
    {: RESULT = crearAtributo("border", crearEstiloBorde(tamanioBorde, tipoBorde,
colorBorde)); :}
;

colores ::=
    RED:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
  | BLUE:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
  | GREEN:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
  | PURPLE:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
  | SKY:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
  | YELLOW:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
```

```
   | BLACK:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
   | WHITE:colorNombrado {: RESULT = crearColorNombrado(colorNombrado); :}
   | RGB_COLOR:colorEspecial {: RESULT = crearColorDesdeTexto(colorEspecial); :}
   | HEX_COLOR:colorEspecial {: RESULT = crearColorDesdeTexto(colorEspecial); :}
   | HSL_COLOR:colorEspecial {: RESULT = crearColorDesdeTexto(colorEspecial); :}
;

fuentes ::= MONO:fuenteTexto {: RESULT = convertirAFuente(fuenteTexto); :}
        | SANS_SERIF:fuenteTexto {: RESULT = convertirAFuente(fuenteTexto); :}
        | CURSIVE:fuenteTexto {: RESULT = convertirAFuente(fuenteTexto); :}
;

bordes ::= LINE:tipoBorde {: RESULT = convertirATipoBorde(tipoBorde); :}
       | DOTTED:tipoBorde {: RESULT = convertirATipoBorde(tipoBorde); :}
       | DOUBLE:tipoBorde {: RESULT = convertirATipoBorde(tipoBorde); :}
;


/* ==================================================
ELEMENTOS
================================================== */

lista_elementos ::= lista_elementos:listaElementosActual COMA
elemento:elementoReducido
            {:
             ArrayList<FormElement> elementosAcumulados =
extraerElementosFormulario(listaElementosActual);
             if (elementoReducido instanceof FormElement)
elementosAcumulados.add((FormElement) elementoReducido);
             RESULT = elementosAcumulados;
            :}
            | elemento:elementoReducido
            {:
             ArrayList<FormElement> elementosIniciales = new ArrayList<>();
             if (elementoReducido instanceof FormElement)
elementosIniciales.add((FormElement) elementoReducido);
             RESULT = elementosIniciales;
            :}
;

elemento ::= seccion:s
        {: RESULT = s; :}
        | tabla:t
        {: RESULT = t; :}
```

```
        | texto:tx
        {: RESULT = tx; :}
        | pregunta:q
        {: RESULT = extraerPreguntaElemento(q); :}
;


/* ==================================================
TABLAS
================================================== */

filas ::= filas:listaFilasActual COMA fila:filaReducida
      {:
        ArrayList<TableRow> filasAcumuladas = extraerFilasTabla(listaFilasActual);
        if (filaReducida instanceof TableRow) filasAcumuladas.add((TableRow) filaReducida);
        RESULT = filasAcumuladas;
      :}
      | fila:filaReducida
      {:
        ArrayList<TableRow> listaFilasInicial = new ArrayList<>();
        if (filaReducida instanceof TableRow) listaFilasInicial.add((TableRow) filaReducida);
        RESULT = listaFilasInicial;
      :}
;

fila ::= COR_IZQ lista_celdas:cells COR_DER
{:
  ArrayList<TableCell> celdasFila = new ArrayList<>();
  if (cells instanceof List<?>) {
    for (Object celdaCruda : (List<?>) cells) {
      if (celdaCruda instanceof TableCell) celdasFila.add((TableCell) celdaCruda);
    }
  }
  RESULT = new TableRow(celdasFila);
:}
;

lista_celdas ::= lista_celdas:listaCeldasActual COMA celda:celdaReducida
            {:
              ArrayList<TableCell> celdasAcumuladas = new ArrayList<>();
              if (listaCeldasActual instanceof List<?>) {
                for (Object celdaCruda : (List<?>) listaCeldasActual) {
                  if (celdaCruda instanceof TableCell) celdasAcumuladas.add((TableCell)
celdaCruda);
```

```
            }
           }
           if (celdaReducida instanceof TableCell) celdasAcumuladas.add((TableCell)
celdaReducida);
           RESULT = celdasAcumuladas;
          :}
          | celda:celdaReducida
          {:
           ArrayList<TableCell> listaCeldasInicial = new ArrayList<>();
           if (celdaReducida instanceof TableCell) listaCeldasInicial.add((TableCell)
celdaReducida);
           RESULT = listaCeldasInicial;
          :}
;

celda ::= LLAVE_IZQ elemento:elementoCelda LLAVE_DER
{:
  if (elementoCelda instanceof FormElement) {
    RESULT = new TableCell((FormElement) elementoCelda);
  } else {
    RESULT = null;
  }
:}
;


/* ==================================================
OPCIONES
================================================== */

lista_opciones_min2 ::= cadena_con_emojis:opcionInicial COMA
cadena_con_emojis:segundaOpcion
             {:
               ArrayList<String> opcionesIniciales = new ArrayList<>();
               opcionesIniciales.add(convertirATexto(opcionInicial));
               opcionesIniciales.add(convertirATexto(segundaOpcion));
               RESULT = opcionesIniciales;
             :}
             | lista_opciones_min2:listaOpcionesActual COMA
cadena_con_emojis:opcionReducida
             {:
               ArrayList<String> opcionesAcumuladas = extraerListaTextos(listaOpcionesActual);
               opcionesAcumuladas.add(convertirATexto(opcionReducida));
               RESULT = opcionesAcumuladas;
```

```
            :}
;

argumento_pokemon ::= NUMBER:v
            {: RESULT = v; :}
            | expresion_aritmetica:v
            {: RESULT = v; :}
;

lista_numeros ::= lista_numeros:listaNumerosActual COMA ENTERO:numeroReducido
          {:
            ArrayList<Integer> numerosAcumulados = extraerListaEnteros(listaNumerosActual);
            Integer numeroConvertido = convertirAEntero(numeroReducido);
            if (numeroConvertido != null) numerosAcumulados.add(numeroConvertido);
            RESULT = numerosAcumulados;
          :}
          | ENTERO:numeroReducido
          {:
            ArrayList<Integer> listaNumerosInicial = new ArrayList<>();
            Integer numeroConvertido = convertirAEntero(numeroReducido);
            if (numeroConvertido != null) listaNumerosInicial.add(numeroConvertido);
            RESULT = listaNumerosInicial;
          :}
;


/* ================================================
CONTROL
================================================ */

bloque_if ::= IF PAREN_IZQ expresion_logica PAREN_DER
         LLAVE_IZQ lista_instrucciones LLAVE_DER
         bloque_else
;

bloque_else ::= ELSE IF PAREN_IZQ expresion_logica PAREN_DER
          LLAVE_IZQ lista_instrucciones LLAVE_DER
          bloque_else
         | ELSE LLAVE_IZQ lista_instrucciones LLAVE_DER
         |
;

bloque_while ::= WHILE PAREN_IZQ expresion_logica PAREN_DER
           LLAVE_IZQ lista_instrucciones LLAVE_DER
```

```
;

bloque_do ::= DO LLAVE_IZQ lista_instrucciones LLAVE_DER
          WHILE PAREN_IZQ expresion_logica PAREN_DER
;

bloque_for ::=
  FOR PAREN_IZQ asignacion_numerica PUNTOYCOMA expresion_logica PUNTOYCOMA
asignacion_numerica PAREN_DER
     LLAVE_IZQ lista_instrucciones LLAVE_DER
   | FOR PAREN_IZQ ID:id IN expresion RANGO expresion PAREN_DER
     {:
       prepararVariableFor(id.toString());
     :}
     LLAVE_IZQ lista_instrucciones LLAVE_DER
;


/* ==================================================
EXPRESIONES
================================================== */

expresion ::= expresion_logica:valorExpresion {: RESULT = valorExpresion; :} ;

expresion_logica ::=
     expresion_logica_or:valorLogico {: RESULT = valorLogico; :}
   | expresion_logica_and:valorLogico {: RESULT = valorLogico; :}
   | NOT expresion_logica:valorLogico {: RESULT = negarOperacionLogica(valorLogico); :}
   | expresion_comparacion:valorComparacion {: RESULT = valorComparacion; :}
;

expresion_logica_or ::=
     expresion_logica_or:izquierda OR expresion_comparacion:derecha {: RESULT =
evaluarOperacionLogica(izquierda, derecha, "OR"); :}
   | expresion_comparacion:izquierda OR expresion_comparacion:derecha {: RESULT =
evaluarOperacionLogica(izquierda, derecha, "OR"); :}
;

expresion_logica_and ::=
     expresion_logica_and:izquierda AND expresion_comparacion:derecha {: RESULT =
evaluarOperacionLogica(izquierda, derecha, "AND"); :}
   | expresion_comparacion:izquierda AND expresion_comparacion:derecha {: RESULT =
evaluarOperacionLogica(izquierda, derecha, "AND"); :}
;
```

```
expresion_comparacion ::=
    expresion_aritmetica:izquierda MAYOR expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, ">"); :}
  | expresion_aritmetica:izquierda MENOR expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, "<"); :}
  | expresion_aritmetica:izquierda MAYOR_IGUAL expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, ">="); :}
  | expresion_aritmetica:izquierda MENOR_IGUAL expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, "<="); :}
  | expresion_aritmetica:izquierda IGUAL expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, "=="); :}
  | expresion_aritmetica:izquierda DIFERENTE expresion_aritmetica:derecha {: RESULT =
evaluarComparacion(izquierda, derecha, "!="); :}
  | expresion_aritmetica:valorAritmetico {: RESULT = valorAritmetico; :}
;

expresion_aritmetica ::=
    expresion_aritmetica:izquierda MAS expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "+"); :}
  | expresion_aritmetica:izquierda MENOS expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "-"); :}
  | expresion_aritmetica:izquierda POR expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "*"); :}
  | expresion_aritmetica:izquierda DIV expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "/"); :}
  | expresion_aritmetica:izquierda POTENCIA expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "^"); :}
  | expresion_aritmetica:izquierda MODULO expresion_aritmetica:derecha {: RESULT =
evaluarOperacionAritmetica(izquierda, derecha, "%"); :}
  | MENOS expresion_aritmetica:valorNegado {: RESULT =
evaluarOperacionAritmetica(crearResultadoValor(0.0), valorNegado, "-"); :} %prec UMINUS
  | PAREN_IZQ expresion_aritmetica:valorInterno PAREN_DER {: RESULT = valorInterno; :}
  | valor:valorReducido {: RESULT = valorReducido; :}
;


/* ==================================================
VALORES
================================================== */

valor ::=
    ENTERO:numeroEntero {: RESULT =
crearResultadoValor(convertirADouble(numeroEntero)); :}
```

```
   | DECIMAL:numeroDecimal {: RESULT =
crearResultadoValor(convertirADouble(numeroDecimal)); :}
  | CADENA:cadenaTexto {: RESULT =
crearResultadoValor(normalizarLiteralCadena(cadenaTexto)); :}
   | ID:id {: RESULT = resolverValorVariable(id.toString()); :}
   | WILDCARD {: RESULT = crearResultadoConComodin(); :}
;


/* ==================================================
CADENA CON EMOJIS
================================================== */

cadena_con_emojis ::= CADENA:value
            {: RESULT = normalizarLiteralCadena(value); :}
            | ID:id
            {: RESULT = convertirATexto(resolverValorVariable(id.toString())); :}
            | cadena_con_emojis:base MAS expresion_aritmetica:valor
            {: RESULT = convertirATexto(base) + convertirATexto(valor); :}
;
```

PKM_PARSER

```
package com.cunoc.compiforms.pkm;

import java_cup.runtime.*;
import com.cunoc.compiforms.form.model.*;
import com.cunoc.compiforms.form.model.elements.*;
import com.cunoc.compiforms.form.model.questions.*;
import com.cunoc.compiforms.form.model.styles.*;
import java.util.*;

parser code {:
  private static final String PKM_TEXT_PREFIX = "__PKM_TEXT__::";
  private boolean hasSyntaxErrors = false;

  public boolean hasSyntaxErrors() {
    return hasSyntaxErrors;
  }

  private int parseIntSafe(String raw) {
    try {
      return Integer.parseInt(raw);
    } catch (NumberFormatException ex) {
      return (int) Double.parseDouble(raw);
    }
  }

  private Double parseDimensionOrNull(String raw) {
    double value = Double.parseDouble(raw);
    return value <= 0 ? null : value;
  }

  private FormElement createOpenOrTextElement(String w, String h, String content, StyleSet
style) {
    LayoutProps layout = new LayoutProps(parseDimensionOrNull(w),
parseDimensionOrNull(h), null, null);
    if (content != null && content.startsWith(PKM_TEXT_PREFIX)) {
      return new TextElement(layout, content.substring(PKM_TEXT_PREFIX.length()), style);
    }
    return new OpenQuestionElement(layout, content, style);
  }

    public void syntax_error(Symbol s) {
      hasSyntaxErrors = true;
        System.err.println("Error sintáctico en Línea " + (s.left + 1) + " Columna " + (s.right + 1) + ".
```

```
No se esperaba este componente: " + s.value);
  }

  public void unrecovered_syntax_error(Symbol s) throws java.lang.Exception {
    hasSyntaxErrors = true;
      System.err.println("Error fatal: No se pudo recuperar del error sintáctico.");
  }
:};

/* Terminales */
terminal TRIPLE_HASH, HASH, OPEN_END_TAG, CLOSE_SELF_TAG, LT, GT;
terminal LBRACE, RBRACE, LBRACK, RBRACK, LPAREN, RPAREN, EQUALS, COLON, COMMA;
terminal AUTHOR, FECHA, HORA, DESCRIPTION, TOTAL, DE, SECCIONES, PREGUNTAS,
ABIERTAS, DESPLEGABLES, SELECCION, MULTIPLES;
terminal SECTION, TABLE, OPEN, DROP, SELECT, MULTIPLE, STYLE, CONTENT, LINE,
ELEMENT;
terminal COLOR_ATTR, BACKGROUND_ATTR, FONT_ATTR, FAMILY_ATTR, TEXT_ATTR,
SIZE_ATTR, BORDER_ATTR;
terminal DOTTED, DOUBLE;
terminal java.lang.String NUMBER_VAL, STRING_VAL, HEX_COLOR, COLOR_NAME,
FONT_VAL, ORIENTATION_VAL, IDENTIFIER;
terminal WHO_IS_THAT_POKEMON;
terminal ERROR;

/* No Terminales */
non terminal FormDocument pkm_file;
non terminal metadata, metadata_entries;
non terminal StyleSet styles_block, style_list, style_item;
non terminal List<FormElement> elements_list;
non terminal List<FormElement> elements_list_non_empty;
non terminal FormElement element;
non terminal SectionElement section_item;
non terminal TableElement table_item;
non terminal FormElement question_item;
non terminal section_header;
non terminal List<FormElement> content_block;
non terminal List<TableRow> line_list;
non terminal TableRow line_item;
non terminal List<TableCell> table_elements_list;
non terminal TableCell table_element;
non terminal ColorValue color_val;
non terminal BorderStyle border_spec;
non terminal OptionsSource options_source;
non terminal List<String> options_list, string_list;
```

```
non terminal List<Integer> correct_val, number_list;
non terminal BorderType border_type_val;

start with pkm_file;

pkm_file ::= metadata
        {: RESULT = new FormDocument(new ArrayList<FormElement>()); :}
        | metadata elements_list_non_empty:el
        {: RESULT = new FormDocument(el); :}
        | metadata styles_block elements_list:el
        {: RESULT = new FormDocument(el); :}
        ;

/* Metadatos */
metadata ::= TRIPLE_HASH metadata_entries TRIPLE_HASH;

metadata_entries ::= AUTHOR COLON STRING_VAL
                FECHA COLON STRING_VAL
                HORA COLON STRING_VAL
                DESCRIPTION COLON STRING_VAL
                TOTAL DE SECCIONES COLON NUMBER_VAL
                TOTAL DE PREGUNTAS COLON NUMBER_VAL
                ABIERTAS COLON NUMBER_VAL
                DESPLEGABLES COLON NUMBER_VAL
                SELECCION COLON NUMBER_VAL
                MULTIPLES COLON NUMBER_VAL;

/* Bloque de Estilos */
styles_block ::= LT STYLE GT style_list:sl OPEN_END_TAG STYLE GT
            {: RESULT = sl; :};

style_list ::= style_list:sl style_item:si
        {: RESULT = si.mergeWith(sl); :}
        | style_item:si
        {: RESULT = si; :};

style_item ::= LT COLOR_ATTR EQUALS color_val:cv CLOSE_SELF_TAG
        {: RESULT = new StyleSet(cv, null, null, null, null); :}
        | LT BACKGROUND_ATTR COLOR_ATTR EQUALS color_val:cv CLOSE_SELF_TAG
        {: RESULT = new StyleSet(null, cv, null, null, null); :}
        | LT FONT_ATTR FAMILY_ATTR EQUALS FONT_VAL:fv CLOSE_SELF_TAG
        {: RESULT = new StyleSet(null, null, FontFamily.valueOf(fv), null, null); :}
        | LT TEXT_ATTR SIZE_ATTR EQUALS NUMBER_VAL:nv CLOSE_SELF_TAG
        {: RESULT = new StyleSet(null, null, null, Double.parseDouble(nv), null); :}
```

```
        | LT BORDER_ATTR COMMA border_spec:bs CLOSE_SELF_TAG
        {: RESULT = new StyleSet(null, null, null, null, bs); :};

color_val ::= HEX_COLOR:hc
        {: RESULT = new ColorValue.Hex(hc); :}
        | COLOR_NAME:cn
        {: RESULT = new ColorValue.Named(NamedColor.valueOf(cn)); :}
        | LPAREN NUMBER_VAL:r COMMA NUMBER_VAL:g COMMA NUMBER_VAL:b RPAREN
        {: RESULT = new ColorValue.Rgb(parseIntSafe(r), parseIntSafe(g), parseIntSafe(b)); :}
        | LT NUMBER_VAL:h COMMA NUMBER_VAL:s COMMA NUMBER_VAL:l GT
        {: RESULT = new ColorValue.Hsl(parseIntSafe(h), parseIntSafe(s), parseIntSafe(l)); :};

border_spec ::= NUMBER_VAL:sz COMMA border_type_val:bt COMMA color_val:cv
        {: RESULT = new BorderStyle(Double.parseDouble(sz), bt, cv); :};

border_type_val ::= LINE {: RESULT = BorderType.LINE; :} | DOTTED {: RESULT =
BorderType.DOTTED; :} | DOUBLE {: RESULT = BorderType.DOUBLE; :};

/* Elementos Principales */
elements_list ::= elements_list:list element:e
            {: list.add(e); RESULT = list; :}
            | /* vacío */
            {: RESULT = new ArrayList<FormElement>(); :} ;

elements_list_non_empty ::= elements_list_non_empty:list element:e
                {: list.add(e); RESULT = list; :}
                | element:e
                {: List<FormElement> l = new ArrayList<>(); l.add(e); RESULT = l; :} ;

element ::= section_item:s {: RESULT = s; :}
        | table_item:t {: RESULT = t; :}
        | question_item:q {: RESULT = q; :}
        | HASH element:e {: RESULT = e; :}
        | ERROR
        {: hasSyntaxErrors = true; RESULT = new TextElement(new LayoutProps(null, null, null,
null), "", null); :};

/* Secciones */
section_item ::= section_header
            | LT SECTION EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
NUMBER_VAL:x COMMA NUMBER_VAL:y COMMA ORIENTATION_VAL:o GT
            content_block:cb
            OPEN_END_TAG SECTION GT
            {: RESULT = new SectionElement(new LayoutProps(Double.parseDouble(w),
```

```
Double.parseDouble(h), Double.parseDouble(x), Double.parseDouble(y)),
Orientacion.valueOf(o), cb, null); :}
        | LT SECTION EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
NUMBER_VAL:x COMMA NUMBER_VAL:y COMMA ORIENTATION_VAL:o GT
          styles_block:sb
          content_block:cb
          OPEN_END_TAG SECTION GT
          {: RESULT = new SectionElement(new LayoutProps(Double.parseDouble(w),
Double.parseDouble(h), Double.parseDouble(x), Double.parseDouble(y)),
Orientacion.valueOf(o), cb, sb); :};

section_header ::= TRIPLE_HASH
            LT SECTION EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
NUMBER_VAL:x COMMA NUMBER_VAL:y COMMA ORIENTATION_VAL:o GT
            OPEN_END_TAG SECTION GT
            TRIPLE_HASH
            {: RESULT = new SectionElement(new LayoutProps(Double.parseDouble(w),
Double.parseDouble(h), Double.parseDouble(x), Double.parseDouble(y)),
Orientacion.valueOf(o), new ArrayList<FormElement>(), null); :};

content_block ::= LT CONTENT GT elements_list:el OPEN_END_TAG CONTENT GT
          {: RESULT = el; :};

/* Tablas */
table_item ::= LT TABLE GT LT CONTENT GT line_list:ll OPEN_END_TAG CONTENT GT
OPEN_END_TAG TABLE GT
        {: RESULT = new TableElement(new LayoutProps(null, null, null, null), ll, null); :}
        | LT TABLE GT styles_block:sb LT CONTENT GT line_list:ll OPEN_END_TAG CONTENT GT
OPEN_END_TAG TABLE GT
        {: RESULT = new TableElement(new LayoutProps(null, null, null, null), ll, sb); :};

line_list ::= line_list:list line_item:li
        {: list.add(li); RESULT = list; :}
        | line_item:li
        {: List<TableRow> l = new ArrayList<>(); l.add(li); RESULT = l; :};

line_item ::= LT LINE GT table_elements_list:tel OPEN_END_TAG LINE GT
        {: RESULT = new TableRow(tel); :};

table_elements_list ::= table_elements_list:list table_element:te
              {: list.add(te); RESULT = list; :}
              | table_element:te
              {: List<TableCell> l = new ArrayList<>(); l.add(te); RESULT = l; :};
```

```
table_element ::= LT ELEMENT GT element:e OPEN_END_TAG ELEMENT GT
        {: RESULT = new TableCell(e); :};

/* Preguntas y Texto */
question_item ::= LT OPEN EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
STRING_VAL:s CLOSE_SELF_TAG
        {: RESULT = createOpenOrTextElement(w, h, s, null); :}
        | LT OPEN EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA STRING_VAL:s
GT styles_block:sb OPEN_END_TAG OPEN GT
        {: RESULT = createOpenOrTextElement(w, h, s, sb); :}
        | LT DROP EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA STRING_VAL:s
COMMA options_source:os COMMA NUMBER_VAL:c CLOSE_SELF_TAG
        {: RESULT = new DropQuestionElement(new LayoutProps(parseDimensionOrNull(w),
parseDimensionOrNull(h), null, null), s, os, parseIntSafe(c), null); :}
        | LT DROP EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA STRING_VAL:s
COMMA options_source:os COMMA NUMBER_VAL:c GT styles_block:sb OPEN_END_TAG
DROP GT
        {: RESULT = new DropQuestionElement(new LayoutProps(parseDimensionOrNull(w),
parseDimensionOrNull(h), null, null), s, os, parseIntSafe(c), sb); :}
        | LT SELECT EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
STRING_VAL:s COMMA options_list:ol COMMA NUMBER_VAL:c CLOSE_SELF_TAG
        {: RESULT = new SelectQuestionElement(new LayoutProps(parseDimensionOrNull(w),
parseDimensionOrNull(h), null, null), ol, parseIntSafe(c), null); :}
        | LT SELECT EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
STRING_VAL:s COMMA options_list:ol COMMA NUMBER_VAL:c GT styles_block:sb
OPEN_END_TAG SELECT GT
        {: RESULT = new SelectQuestionElement(new LayoutProps(parseDimensionOrNull(w),
parseDimensionOrNull(h), null, null), ol, parseIntSafe(c), sb); :}
        | LT MULTIPLE EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
STRING_VAL:s COMMA options_list:ol COMMA correct_val:cv CLOSE_SELF_TAG
        {: RESULT = new MultipleQuestionElement(new
LayoutProps(parseDimensionOrNull(w), parseDimensionOrNull(h), null, null), ol, cv, null); :}
        | LT MULTIPLE EQUALS NUMBER_VAL:w COMMA NUMBER_VAL:h COMMA
STRING_VAL:s COMMA options_list:ol COMMA correct_val:cv GT styles_block:sb
OPEN_END_TAG MULTIPLE GT
        {: RESULT = new MultipleQuestionElement(new
LayoutProps(parseDimensionOrNull(w), parseDimensionOrNull(h), null, null), ol, cv, sb); :};

options_source ::= options_list:ol
         {: RESULT = new OptionsSource.Static(ol); :}
         | WHO_IS_THAT_POKEMON LPAREN IDENTIFIER:selector COMMA
NUMBER_VAL:inicio COMMA NUMBER_VAL:end RPAREN
         {: RESULT = new OptionsSource.PokemonRange(selector, parseIntSafe(inicio),
parseIntSafe(end)); :};
```

```
options_list ::= LBRACE string_list:sl RBRACE {: RESULT = sl; :}
        | LBRACK string_list:sl RBRACK {: RESULT = sl; :};

string_list ::= string_list:list COMMA STRING_VAL:s
        {: list.add(s); RESULT = list; :}
        | STRING_VAL:s
        {: List<String> l = new ArrayList<>(); l.add(s); RESULT = l; :};

correct_val ::= LBRACE number_list:nl RBRACE {: RESULT = nl; :}
        | LBRACK number_list:nl RBRACK {: RESULT = nl; :}
        | LBRACE RBRACE {: RESULT = new ArrayList<Integer>(); :}
        | LBRACK RBRACK {: RESULT = new ArrayList<Integer>(); :};

number_list ::= number_list:list COMMA NUMBER_VAL:n
        {: list.add(parseIntSafe(n)); RESULT = list; :}
        | NUMBER_VAL:n
        {: List<Integer> l = new ArrayList<>(); l.add(parseIntSafe(n)); RESULT = l; :};
```

Lexer

```
package com.cunoc.compiforms;

import java_cup.runtime.Symbol;
import java.util.ArrayList;
import java.util.List;
import com.cunoc.compiforms.tokens.*;

%%

%public
%unicode
%class Lexer
%cup
%line
%column
%char

%{

  private ArrayList<TokenInfo> tokens = new ArrayList<>();
  private ArrayList<ErrorInfo> errors = new ArrayList<>();

  public List<TokenInfo> getTokens() {
     return tokens;
  }

  public List<ErrorInfo> getErrors() {
     return errors;
  }

  private Symbol symbol(int type, TokenType visualType) {
    int start = (int) yychar;
    int end = (int) (yychar + yylength());

    tokens.add(new TokenInfo(visualType, yytext(), start, end, yyline + 1, yycolumn + 1));

    return new Symbol(type, yyline, yycolumn, yytext());
  }

  private void addLexicalError(String lexeme, int line, int column) {
     errors.add(new ErrorInfo(ErrorType.LEXICAL, lexeme, line + 1, column + 1, "Símbolo no
reconocido en el lenguaje"));
  }
```

```
%}

/*
==============================================================
|                    MACROS                    |
==============================================================
*/

DIGITO        = [0-9]
LETRA         = [a-zA-Z_]
ID            = {LETRA}({LETRA}|{DIGITO})*
ENTERO        = {DIGITO}+
DECIMAL       = {DIGITO}+"."{DIGITO}+
ESPACIO       = [ \t\r\n\f]+
CADENA        = \"([^\\\"\n]|\\.)*\"
COMMENT_LINEA   = \$[^\n]*
COMMENT_BLOQUE  = "/*"([^*]|\*+[^*/])*\*+"/"

HEX_COLOR      = \#[0-9a-fA-F]{6}
RGB_COLOR      = (rgb)?\([ \t]*{DIGITO}+[ \t]*,[ \t]*{DIGITO}+[ \t]*,[ \t]*{DIGITO}+[ \t]*\)
HSL_COLOR      = (<[ \t]*{DIGITO}+[ \t]*,[ \t]*{DIGITO}+[ \t]*,[ \t]*{DIGITO}+[ \t]*>)|(hsl\([
\t]*{DIGITO}+%?[ \t]*,[ \t]*{DIGITO}+%?[ \t]*,[ \t]*{DIGITO}+%?[ \t]*\))

/* Emojis del enunciado */
EMOJI_SMILE_FACE       = @\[:\)+\]
EMOJI_SMILE_WORD       = @\[:smile:\]
EMOJI_SAD_FACE         = @\[:\(+\]
EMOJI_SAD_WORD         = @\[:sad:\]
EMOJI_SERIOUS_FACE     = @\[:\|+\]
EMOJI_SERIOUS_WORD     = @\[:serious:\]
EMOJI_HEART_FACE       = @\[<+3+\]
EMOJI_HEART_WORD       = @\[:heart:\]
EMOJI_CAT_FACE         = @\[:\^+:\]
EMOJI_CAT_WORD         = @\[:cat:\]
EMOJI_STAR             = @\[:star:\]
EMOJI_STAR_COUNT_COLON  = @\[:star:({DIGITO}+|number):\]
EMOJI_STAR_COUNT_DASH   = @\[:star-({DIGITO}+|number):\]
EMOJI =
({EMOJI_SMILE_FACE}|{EMOJI_SMILE_WORD}|{EMOJI_SAD_FACE}|{EMOJI_SAD_WORD}|{EMOJ
I_SERIOUS_FACE}|{EMOJI_SERIOUS_WORD}|{EMOJI_HEART_FACE}|{EMOJI_HEART_WORD}|{E
MOJI_CAT_FACE}|{EMOJI_CAT_WORD}|{EMOJI_STAR}|{EMOJI_STAR_COUNT_COLON}|{EMOJI_
STAR_COUNT_DASH})
```

```
%%

/*
=================================================================
|                PALABRAS RESERVADAS                |
=================================================================
*/

    // Estructuras
    "SECTION"           { return symbol(sym.SECTION, TokenType.KEYWORD); }
    "TABLE"             { return symbol(sym.TABLE, TokenType.KEYWORD); }
    "TEXT"              { return symbol(sym.TEXT, TokenType.KEYWORD); }

    // Preguntas
    "OPEN_QUESTION"       { return symbol(sym.OPEN_QUESTION, TokenType.KEYWORD); }
    "DROP_QUESTION"       { return symbol(sym.DROP_QUESTION, TokenType.KEYWORD); }
    "SELECT_QUESTION"     { return symbol(sym.SELECT_QUESTION, TokenType.KEYWORD); }
    "MULTIPLE_QUESTION"    { return symbol(sym.MULTIPLE_QUESTION,
TokenType.KEYWORD); }

    // Control
    "IF"                { return symbol(sym.IF, TokenType.KEYWORD); }
    "ELSE"              { return symbol(sym.ELSE, TokenType.KEYWORD); }
    "WHILE"             { return symbol(sym.WHILE, TokenType.KEYWORD); }
    "DO"                { return symbol(sym.DO, TokenType.KEYWORD); }
    "FOR"               { return symbol(sym.FOR, TokenType.KEYWORD); }
    "IN"                { return symbol(sym.IN, TokenType.KEYWORD); }
    "in"                { return symbol(sym.IN, TokenType.KEYWORD); }

    // Definición de variables
    "number"            { return symbol(sym.NUMBER, TokenType.KEYWORD); }
    "string"            { return symbol(sym.STRING, TokenType.KEYWORD); }
    "special"           { return symbol(sym.SPECIAL, TokenType.KEYWORD); }

    // Orientación
    "VERTICAL"          { return symbol(sym.VERTICAL, TokenType.KEYWORD); }
    "HORIZONTAL"        { return symbol(sym.HORIZONTAL, TokenType.KEYWORD); }

    // Fuentes
    "MONO"              { return symbol(sym.MONO, TokenType.KEYWORD); }
    "SANS_SERIF"        { return symbol(sym.SANS_SERIF, TokenType.KEYWORD); }
    "CURSIVE"           { return symbol(sym.CURSIVE, TokenType.KEYWORD); }

    // Bordes
```

```
    "LINE"            { return symbol(sym.LINE, TokenType.KEYWORD); }
    "DOTTED"          { return symbol(sym.DOTTED, TokenType.KEYWORD); }
    "DOUBLE"          { return symbol(sym.DOUBLE, TokenType.KEYWORD); }

    // Colores predefinidos
    "RED"             { return symbol(sym.RED, TokenType.KEYWORD); }
    "BLUE"            { return symbol(sym.BLUE, TokenType.KEYWORD); }
    "GREEN"           { return symbol(sym.GREEN, TokenType.KEYWORD); }
    "PURPLE"          { return symbol(sym.PURPLE, TokenType.KEYWORD); }
    "SKY"             { return symbol(sym.SKY, TokenType.KEYWORD); }
    "YELLOW"          { return symbol(sym.YELLOW, TokenType.KEYWORD); }
    "BLACK"           { return symbol(sym.BLACK, TokenType.KEYWORD); }
    "WHITE"           { return symbol(sym.WHITE, TokenType.KEYWORD); }

    // Funciones especiales
    "who_is_that_pokemon"  { return symbol(sym.WHO_IS_THAT_POKEMON,
TokenType.KEYWORD); }
    "draw"            { return symbol(sym.DRAW, TokenType.KEYWORD); }

    // Atributos
    "width"           { return symbol(sym.WIDTH, TokenType.KEYWORD); }
    "height"          { return symbol(sym.HEIGHT, TokenType.KEYWORD); }
    "label"           { return symbol(sym.LABEL, TokenType.KEYWORD); }
    "orientation"     { return symbol(sym.ORIENTATION, TokenType.KEYWORD); }
    "elements"        { return symbol(sym.ELEMENTS, TokenType.KEYWORD); }
    "pointX"          { return symbol(sym.POINTX, TokenType.KEYWORD); }
    "pointY"          { return symbol(sym.POINTY, TokenType.KEYWORD); }
    "content"         { return symbol(sym.CONTENT, TokenType.KEYWORD); }
    "options"         { return symbol(sym.OPTIONS, TokenType.KEYWORD); }
    "correct"         { return symbol(sym.CORRECT, TokenType.KEYWORD); }

    // Estilos con espacio obligatorio entre comillas
    "\"color\""       { return symbol(sym.COLOR, TokenType.KEYWORD); }
    "\"background color\""  { return symbol(sym.BACKGROUND_COLOR,
TokenType.KEYWORD); }
    "\"font family\""     { return symbol(sym.FONT_FAMILY, TokenType.KEYWORD); }
    "\"text size\""       { return symbol(sym.TEXT_SIZE, TokenType.KEYWORD); }
    "\"border\""          { return symbol(sym.BORDER, TokenType.KEYWORD); }
    "styles"          { return symbol(sym.STYLES, TokenType.KEYWORD); }

/*
============================================================
|            OPERADORES ARITMETICOS            |
============================================================
```

```
*/

"+"             { return symbol(sym.MAS, TokenType.OPERATOR); }
"-"             { return symbol(sym.MENOS, TokenType.OPERATOR); }
"*"             { return symbol(sym.POR, TokenType.OPERATOR); }
"/"             { return symbol(sym.DIV, TokenType.OPERATOR); }
"^"             { return symbol(sym.POTENCIA, TokenType.OPERATOR); }
"%"             { return symbol(sym.MODULO, TokenType.OPERATOR); }

"("             { return symbol(sym.PAREN_IZQ, TokenType.SYMBOL); }
")"             { return symbol(sym.PAREN_DER, TokenType.SYMBOL); }

/*
=============================================================
|            OPERADORES DE COMPARACION            |
=============================================================
*/

"=="            { return symbol(sym.IGUAL, TokenType.OTHER); }
"!="            { return symbol(sym.DIFERENTE, TokenType.OTHER); }
"!!"            { return symbol(sym.DIFERENTE, TokenType.OTHER); }
">="            { return symbol(sym.MAYOR_IGUAL, TokenType.OTHER); }
"<="            { return symbol(sym.MENOR_IGUAL, TokenType.OTHER); }
">"             { return symbol(sym.MAYOR, TokenType.OTHER); }
"<"             { return symbol(sym.MENOR, TokenType.OTHER); }

/*
=============================================================
|              OPERADORES LOGICOS                 |
=============================================================
*/

"&&"            { return symbol(sym.AND, TokenType.OTHER); }
"||"            { return symbol(sym.OR, TokenType.OTHER); }
"~"             { return symbol(sym.NOT, TokenType.OTHER); }

"="             { return symbol(sym.ASIGNACION, TokenType.OPERATOR); }

/*
=============================================================
|                SIMBOLOS                         |
=============================================================
*/
```

```
"{"             { return symbol(sym.LLAVE_IZQ, TokenType.SYMBOL); }
"}"             { return symbol(sym.LLAVE_DER, TokenType.SYMBOL); }
"["             { return symbol(sym.COR_IZQ, TokenType.SYMBOL); }
"]"             { return symbol(sym.COR_DER, TokenType.SYMBOL); }

":"             { return symbol(sym.DOS_PUNTOS, TokenType.SYMBOL); }
","            { return symbol(sym.COMA, TokenType.SYMBOL); }
";"             { return symbol(sym.PUNTOYCOMA, TokenType.SYMBOL); }
".."           { return symbol(sym.RANGO, TokenType.SYMBOL); }
"."            { return symbol(sym.PUNTO, TokenType.SYMBOL); }

"?"             { return symbol(sym.WILDCARD, TokenType.OTHER); }

/*
============================================================
|              LITERALES                    |
============================================================
*/

{DECIMAL}          { return symbol(sym.DECIMAL, TokenType.NUMBER); }
{ENTERO}           { return symbol(sym.ENTERO, TokenType.NUMBER); }
{CADENA}           { return symbol(sym.CADENA, TokenType.STRING); }
{ID}            { return symbol(sym.ID, TokenType.VARIABLE); }

{HEX_COLOR}        { return symbol(sym.HEX_COLOR, TokenType.KEYWORD); }
{RGB_COLOR}        { return symbol(sym.RGB_COLOR, TokenType.KEYWORD); }
{HSL_COLOR}        { return symbol(sym.HSL_COLOR, TokenType.KEYWORD); }

{EMOJI}            { return symbol(sym.EMOJI, TokenType.EMOJI); }

/*
============================================================
|              COMENTARIOS                  |
============================================================
*/

{COMMENT_LINEA}      { /* ignora comentarios */ }
{COMMENT_BLOQUE}     { /* ignora comentarios */ }

{ESPACIO}          { /* ignora espacios */ }

/* ----------- ERROR LEXICO ----------- */

. {
```

```
    addLexicalError(yytext(), yyline, yycolumn);
    return symbol(sym.ERROR, TokenType.ERROR);
}
```

PKM_LEXER

```
package com.cunoc.compiforms.pkm;

import java_cup.runtime.Symbol;

%%

%public
%unicode
%class PKMLexer
%cup
%cupsym pkm_sym
%line
%column
%char
%ignorecase
%eofval{
  return new java_cup.runtime.Symbol(pkm_sym.EOF);
%eofval}

%{
  private Symbol symbol(int type) {
    return new Symbol(type, yyline, yycolumn);
  }
  private Symbol symbol(int type, Object value) {
    return new Symbol(type, yyline, yycolumn, value);
  }
%}

/* Expresiones Regulares */
WhiteSpace = [ \t\r\n\f]+
Digit      = [0-9]
Number     = -?{Digit}+(\.{Digit}+)?
String     = \"([^\"\\]|\\.)*\"
HexColor   = #[0-9a-fA-F]{6}
Identifier = [A-Za-z_][A-Za-z0-9_]*

%%

<YYINITIAL> {
  {WhiteSpace}   { /* Ignorar espacios */ }

  /* Palabras Reservadas de Metadatos (Normalizadas a minÃºsculas en el valor) */
  "Author"      { return symbol(pkm_sym.AUTHOR, yytext().toLowerCase()); }
```

```
  "Fecha"      { return symbol(pkm_sym.FECHA, yytext().toLowerCase()); }
  "Hora"       { return symbol(pkm_sym.HORA, yytext().toLowerCase()); }
  "Description"  { return symbol(pkm_sym.DESCRIPTION, yytext().toLowerCase()); }
  "Total"      { return symbol(pkm_sym.TOTAL, yytext().toLowerCase()); }
  "de"         { return symbol(pkm_sym.DE, yytext().toLowerCase()); }
  "Secciones"   { return symbol(pkm_sym.SECCIONES, yytext().toLowerCase()); }
  "Preguntas"   { return symbol(pkm_sym.PREGUNTAS, yytext().toLowerCase()); }
  "Abiertas"    { return symbol(pkm_sym.ABIERTAS, yytext().toLowerCase()); }
  "Desplegables" { return symbol(pkm_sym.DESPLEGABLES, yytext().toLowerCase()); }
  "Selección"   { return symbol(pkm_sym.SELECCION, yytext().toLowerCase()); }
  "Múltiples"   { return symbol(pkm_sym.MULTIPLES, yytext().toLowerCase()); }

  /* Funciones reservadas */
  "WHO_IS_THAT_POKEMON" { return symbol(pkm_sym.WHO_IS_THAT_POKEMON,
yytext().toUpperCase()); }

  /* Palabras Reservadas de Estructura */
  "Section"     { return symbol(pkm_sym.SECTION, yytext().toLowerCase()); }
  "Table"      { return symbol(pkm_sym.TABLE, yytext().toLowerCase()); }
  "Open"       { return symbol(pkm_sym.OPEN, yytext().toLowerCase()); }
  "Drop"       { return symbol(pkm_sym.DROP, yytext().toLowerCase()); }
  "Select"     { return symbol(pkm_sym.SELECT, yytext().toLowerCase()); }
  "Multiple"    { return symbol(pkm_sym.MULTIPLE, yytext().toLowerCase()); }
  "Style"      { return symbol(pkm_sym.STYLE, yytext().toLowerCase()); }
  "Content"     { return symbol(pkm_sym.CONTENT, yytext().toLowerCase()); }
  "Line"       { return symbol(pkm_sym.LINE, yytext().toLowerCase()); }
  "Element"     { return symbol(pkm_sym.ELEMENT, yytext().toLowerCase()); }

  /* OrientaciÃ³n y Constantes */
  "VERTICAL"    { return symbol(pkm_sym.ORIENTATION_VAL, "VERTICAL"); }
  "HORIZONTAL"   { return symbol(pkm_sym.ORIENTATION_VAL, "HORIZONTAL"); }

  /* Atributos de Estilo */
  "color"      { return symbol(pkm_sym.COLOR_ATTR, yytext().toLowerCase()); }
  "background"   { return symbol(pkm_sym.BACKGROUND_ATTR, yytext().toLowerCase()); }
  "font"       { return symbol(pkm_sym.FONT_ATTR, yytext().toLowerCase()); }
  "family"      { return symbol(pkm_sym.FAMILY_ATTR, yytext().toLowerCase()); }
  "text"       { return symbol(pkm_sym.TEXT_ATTR, yytext().toLowerCase()); }
  "size"       { return symbol(pkm_sym.SIZE_ATTR, yytext().toLowerCase()); }
  "border"      { return symbol(pkm_sym.BORDER_ATTR, yytext().toLowerCase()); }

  /* Colores Predefinidos */
  "RED"        { return symbol(pkm_sym.COLOR_NAME, "RED"); }
  "BLUE"       { return symbol(pkm_sym.COLOR_NAME, "BLUE"); }
```

```
  "GREEN"      { return symbol(pkm_sym.COLOR_NAME, "GREEN"); }
  "PURPLE"     { return symbol(pkm_sym.COLOR_NAME, "PURPLE"); }
  "SKY"        { return symbol(pkm_sym.COLOR_NAME, "SKY"); }
  "YELLOW"     { return symbol(pkm_sym.COLOR_NAME, "YELLOW"); }
  "BLACK"      { return symbol(pkm_sym.COLOR_NAME, "BLACK"); }
  "WHITE"      { return symbol(pkm_sym.COLOR_NAME, "WHITE"); }

  /* Tipos de Fuente */
  "MONO"       { return symbol(pkm_sym.FONT_VAL, "MONO"); }
  "SANS_SERIF"  { return symbol(pkm_sym.FONT_VAL, "SANS_SERIF"); }
  "CURSIVE"    { return symbol(pkm_sym.FONT_VAL, "CURSIVE"); }

  /* Tipos de Borde (DOTTED y DOUBLE) - LINE ya estÃ¡ arriba */
  "DOTTED"     { return symbol(pkm_sym.DOTTED, "DOTTED"); }
  "DOUBLE"     { return symbol(pkm_sym.DOUBLE, "DOUBLE"); }

  /* SÃmbolos */
  "###"        { return symbol(pkm_sym.TRIPLE_HASH); }
  "#"          { return symbol(pkm_sym.HASH); }
  "</"         { return symbol(pkm_sym.OPEN_END_TAG); }
  "/>"         { return symbol(pkm_sym.CLOSE_SELF_TAG); }
  "<"          { return symbol(pkm_sym.LT); }
  ">"          { return symbol(pkm_sym.GT); }
  "{"          { return symbol(pkm_sym.LBRACE); }
  "}"          { return symbol(pkm_sym.RBRACE); }
  "["          { return symbol(pkm_sym.LBRACK); }
  "]"          { return symbol(pkm_sym.RBRACK); }
  "("          { return symbol(pkm_sym.LPAREN); }
  ")"          { return symbol(pkm_sym.RPAREN); }
  "="          { return symbol(pkm_sym.EQUALS); }
  ":"          { return symbol(pkm_sym.COLON); }
  ","          { return symbol(pkm_sym.COMMA); }

  /* Valores */
  {Number}     { return symbol(pkm_sym.NUMBER_VAL, yytext()); }
  {String}     { return symbol(pkm_sym.STRING_VAL, yytext().substring(1, yytext().length()-1)); }
  {HexColor}   { return symbol(pkm_sym.HEX_COLOR, yytext()); }
  {Identifier}  { return symbol(pkm_sym.IDENTIFIER, yytext()); }

  /* Error LÃ©xico */
  . { return symbol(pkm_sym.ERROR, yytext()); }
}
```